Fibonacci Heap

Outperforming Traditional Priority Queues

Pedro Marinho Miguel Mateus Tuomas Haapasalo

FEUP

March 23, 2026

Overview

Overview

Motivation

- Efficient priority queues are a key problem in the context of graph algorithm design.
- There is an increased need for processing massive networks, but the repeated cost of updating node priorities poses a huge bottleneck!
- Demand for priority queue designs capable of extremely fast updates, but that can simultaneously keep other standard operations efficient.

The $O(\log n)$ Bound

Currently, the effects of logarithmic slowness are much more visible! The number of edges in dense networks vastly exceeds the vertices, making priority updates the dominant computing cost [1].

Overview

The Fibonacci Heap

Definition

The Fibonacci heap is a data structure consisting of a collection, or forest, of min-heap ordered trees, such that:

- Every parent node maintains a key value that is less than or equal to the values of its children
- The absolute minimum element of any individual tree will always be found at its root
- The trees in this forest do not need to maintain a rigid or balanced shape

The Fibonacci Heap

Anatomy of a Node

Each node x in the heap contains several important attributes:

- $x.key$: Priority value of the element
- $x.degree$: Number of direct children the node possesses
- Pointers:
  - Vertical: $x.parent$ and $x.child$ (points to any one of its children)
  - Horizontal: $x.left$ and $x.right$ (connects to adjacent siblings)
- $x.mark$: A boolean value indicating whether the node has lost a child since it was last made the child of another node

The Fibonacci Heap

Circular Doubly Linked Lists

To organize the nodes efficiently, this data structure relies on circular doubly linked lists

- All siblings, which are children of the same parent, are connected to each other in a continuous loop
- The roots of all the trees in the forest are also linked together to form a global root list

Why this matters

This way of organizing allows us to more easily manage and compute operations, such as insertion or key decrease, in an amortized time of $O(1)$.

Overview

Operations - Slide 1

Operations - Slide 2

Operations - Slide 3

Operations - Slide 4

Operations - Slide 5

Overview

Applications - Slide 1

Applications - Slide 2

Applications - Slide 3

Applications - Slide 4

Overview

Example - Slide 1

Example - Slide 2

Example - Slide 3

Example - Slide 4

Overview

Conclusions - Slide 1

Conclusions - Slide 2

Overview

References


Michael L. Fredman and Robert Endre Tarjan.
Fibonacci heaps and their uses in improved network optimization algorithms.
Journal of the ACM, 34(3):596–615, 1987.


Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms, Third Edition.
MIT Press, 2009.